HaqqAI

System design

Trustworthy AI platform for financial enterprises
Version 1.0, September 2026

This document describes how HaqqAI is put together: what runs, in what order, which parts already work and which are still on paper. It is written for a technical reader who has not seen the code. Wherever something is only designed and not built, it says so.

1. The problem the design has to solve

A language model will state a wrong number in exactly the same tone as a right one. Inside a bank that is a compliance problem, so HaqqAI is not built on the hope that the model gets it right. The model writes and reads text. Everything that has to be exact, such as comparing two figures, finding the page a fact came from, or deciding whether an answer is supported, is done by ordinary code. When the evidence does not back a claim, the system declines to answer.

2. The system at a glance

A person asks a question in the web interface. The interface calls an API service, which passes the question to the verification engine. The engine reads from three indexes built ahead of time from the institution's documents, calls a language model at a few fixed points, and returns either a verified answer with its sources or a refusal.

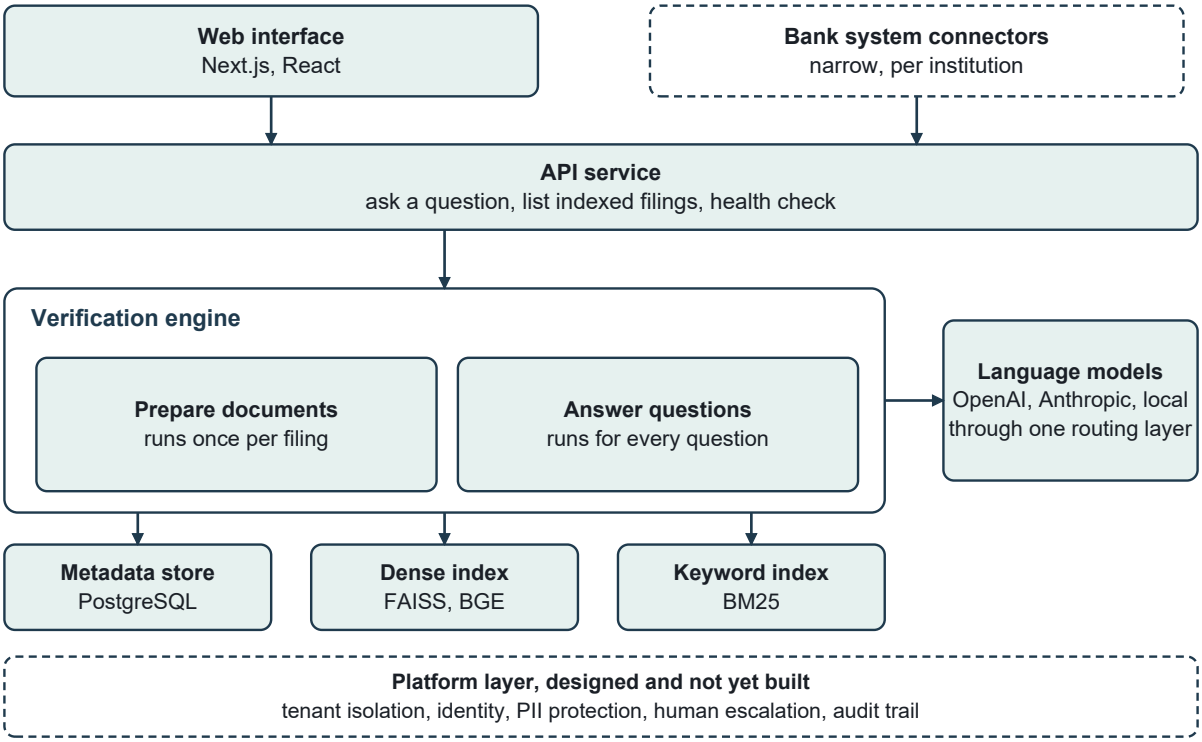


Figure 1. Main parts of the system. Solid boxes are built and tested. Dashed boxes are designed and not yet built.

The verification engine, its indexes, the model routing layer, the API service and the web interface are built. The platform layer around them (tenant isolation, identity, privacy controls, escalation, audit and connectors) is fully designed and is the main work still to do. Section 9 covers it.

3. Design rules

Six rules shape every component. A feature that breaks one of them does not ship.

Rule	What it means in practice
The model only handles language	Models write sentences and read text. Numeric comparison, citations and access decisions are code.
The checker never sees the draft	The step that verifies an answer works from the evidence alone, so it cannot simply agree with the writer.
Decline instead of guessing	Missing evidence, a mismatch or a timeout ends in a refusal. No reply is never treated as approval.
The bank stays in charge of access	The bank's own systems decide who a person is and what they may see. The AI works inside those decisions.
Isolation in layers	Each institution's data is separated in the application, database, vector store, cache and configuration, so one missed filter cannot leak anything.
Choices are configuration	A new bank, model, price or deployment shape is a settings change, not a code change.

4. Preparing documents

Documents are prepared once, before any question is asked. The result is a set of short, self-contained facts, each tied to its source, held in three indexes.

Step	What happens	Uses a model?
Parse	PDF pages are read for text and tables. Scanned pages go through an OCR step. Structured files (XBRL, CSV, Excel) take a separate path.	No
Split into facts	Each page is turned into short statements such as "the company had accounts payable of 2,266 million at December 31, 2018". Every fact keeps its document, page, filing type and filing date. Code drops any statement containing a digit that is not in the source text.	Yes, checked by code
Resolve number details	A second small model step works out qualifiers such as the period a figure belongs to. It never writes digits.	Yes, limited
Register company names	Every name a company appears under is mapped to one placeholder token, so two companies cannot be confused inside a prompt.	No
Index	Each fact is stored three ways: as a vector for meaning, in a keyword index, and as a database record. All three must hold exactly the same set of fact IDs, and a test enforces it.	No

The current index holds 21,913 facts from five filings across three companies: 3M (two annual reports and one quarterly report), Activision Blizzard and Adobe.

5. Answering a question

Every question follows the same fixed sequence. There is no step where a model decides what to do next, which keeps the behaviour predictable and each run fully traceable.

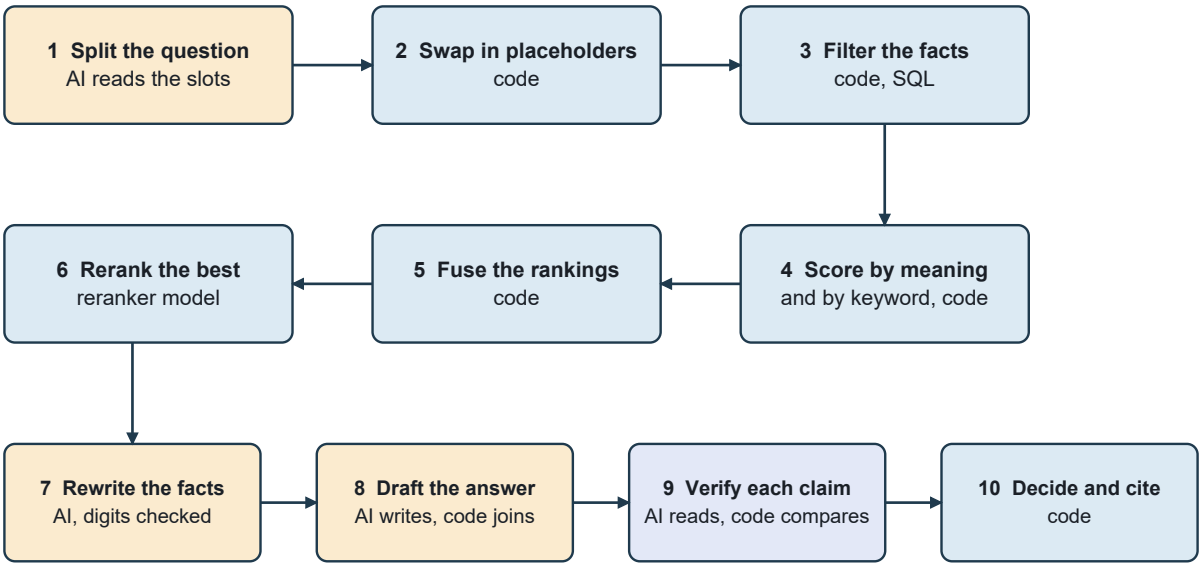


Figure 2. The path of one question. Amber steps call a language model. Lilac steps call one and then have code check the result. Blue steps use no language model.

Step	What happens
1. Split the question	A model reads the question and lists the companies, metrics and periods in it. It does not answer or decompose anything. Code then builds one sub-question for every combination, so a two-metric question always becomes two lookups and nothing can be skipped.
2. Placeholders	Company names are replaced with tokens before any later model call, and put back at the end.
3. Filter	A database query keeps only the facts for the right company, fiscal year and filing type, so a figure from the wrong company or year cannot be retrieved.
4. Score	Remaining facts are scored by meaning (vector similarity) and by keyword (BM25).
5. Fuse	The two rankings are combined by reciprocal rank fusion. A small optional weighting favours newer filings.
6. Rerank	A cross-encoder model reads the question and each candidate together and keeps the best ten of the top twenty.
7. Rewrite	A model rewrites each kept fact as a standalone sentence. Code rejects any rewrite that contains a number not present in the source fact.
8. Draft	A model writes the answer from those facts. Code joins the sub-answers, one line per figure.
9. Verify	The draft is split into individual claims. Each is checked against the evidence by a step that cannot see the draft. Numbers are compared by code, wording by a model.
10. Decide and cite	If any claim conflicts with or lacks support in the evidence, the answer is withheld with a plain message. Otherwise sources are looked up from stored metadata, never written by a model, and company names are restored.

6. Where models are used, and where they are not

The engine has six model call sites in total. Each runs at temperature zero with a pinned model version, and each returns output that is checked against a fixed schema. The current configuration pins a snapshot of gpt-4.1-mini for these calls, with a stronger model kept for the rare page that fails a table completeness check. The routing layer also supports Anthropic models and local models, selected per call site in configuration.

Call site	Job	Limit
Fact segmenter	Turns a page of a filing into short facts	Output digits must appear in the source
Number qualifier	Works out the period and unit a figure belongs to	Never writes a digit
Question parser	Lists companies, metrics and periods in a question	Slot parsing only, the split itself is code
Fact rewriter	Makes each retrieved fact a standalone sentence	Rewrite rejected if it adds a digit
Answer writer	Drafts the answer from the checked facts	May only use the facts it is given
Verifier	Extracts claims, answers them from evidence, compares wording	Never sees the draft, never compares numbers

A model never compares two numbers, never writes a citation, never decides who may see a record and never approves a transaction.

7. Data stores and the rules that keep them consistent

Store	Technology	Holds
Metadata store	PostgreSQL 16, one row per fact	The fact text, company, ticker, filing type and date, section, page and source document. Used for filtering and for looking up citations.
Dense index	FAISS with BGE base vectors (768 dimensions), model version pinned	One vector per fact, for search by meaning.
Keyword index	BM25, standard settings	Term statistics per fact, rebuilt from the metadata store when the service starts.

Three rules hold everywhere. First, the three stores contain the identical set of fact IDs. Second, the fact record layout is frozen, because the verification step depends on it. Third, page numbers are stored zero-indexed and only converted to viewer page numbers at the edge, in the API response. In the multi-tenant design every record also carries a tenant ID, and all three stores key on the pair of tenant and fact.

8. Interfaces

The API service is a FastAPI application with three endpoints. **POST /api/ask** takes a question and returns the status (answered or declined), the answer text, the extracted values, the sources with page numbers and the exact evidence sentence, the verdict on each claim, a verification score and details of how the answer was produced. **GET /api/info** lists the companies and filings that are indexed. **GET /api/health** reports whether the service is up.

The verification score is the share of claims in the draft that matched the evidence, computed in code. It is not a model's opinion of its own confidence.

The web interface is a Next.js application. It shows the question box, example questions, the indexed filings, and for each answer the verdict, the claims and their checks, and the sources. A refusal is shown as a refusal and does not display evidence. The interface reaches the API through its own server-side routes, so the browser never talks to the API service directly.

9. The platform layer (designed, not yet built)

The verification engine is the core of a larger product for a bank. The rest is specified in detail and follows the same rules as the engine.

Product areas

Five areas share the one engine: customer experience, employee experience, banking operations, management, and AI governance. Governance is not a separate module. It is the layer the other four sit on: citation checking, confidence scoring, audit, access control, escalation and monitoring.

Two operating modes

A controlled mode runs the fixed sequence in section 5 and is used for everything customer facing and most staff tools. A research mode allows multi-step work with tools, and is available only to modules explicitly assigned to it, such as regulatory research. The mode belongs to the module and is set when the module is designed. It is never chosen from the wording of a question.

Multi-tenancy and isolation

One codebase serves shared cloud, dedicated infrastructure, and on-premises installs. Isolation is enforced at each layer independently:

Layer	How each institution is kept apart
Application	The tenant is resolved once per request and passed through every call.
Database	Row-level security tied to the connection's tenant.
Vector store	Separate namespaces per tenant.
Files and cache	Separate storage prefixes, and the tenant is part of every cache key.
Configuration, logs, backups	Same enforcement as runtime data. Logs are tenant-tagged. Backups restore one tenant without touching another.

Identity and authorization

Authentication says who someone is. Authorization says what they may do. The AI acts only inside whatever those two have already decided and never answers either question itself. Staff sign in through the bank's own identity provider. Customers go through a configurable authentication flow. Access is checked per object, including for past conversation content, before anything is placed in a model's context.

Privacy

Personal data such as account numbers, BVN and NIN is replaced with tokens before anything is embedded or sent to a model. The real value sits in a short-lived, encrypted, session-scoped vault, and only code at the connector boundary can resolve it. The model never sees the real value. Which categories are redacted, and for how long records are kept, is set by each bank.

Human escalation

A case goes to a person when confidence is below the bank's threshold, when the action is a write or is high risk, when policy requires it, when the verifier declines a material claim, or when the user asks. Cases route to the right queue by role and module. A reviewer can never approve their own request, and the most sensitive categories need two approvers. A timeout is never treated as permission.

Audit trail

Every retrieval, model call, permission check, escalation decision, configuration change and sign-in is recorded against tenant, user and session. The log is append-only and hash-chained, so altering history breaks the chain visibly. No role, including a platform administrator, can delete an audit record.

Connectors and deployment

Each bank system (core banking, HR, CRM, fraud tools) is reached through a narrow, task-specific connector, deployed per institution and never shared, so a compromise stays inside one tenant. Connector credentials and connection details are never visible to the model. The whole platform ships as one container image with three deployment profiles that differ only in configuration: managed cloud, self-hosted or on-premises, and a lean single-server profile.

10. Technology

Area	Choice
Language and services	Python 3.11 or later, FastAPI and Uvicorn, Pydantic for schemas
Web interface	Next.js 16, React 19, TypeScript, Tailwind CSS
Metadata store	PostgreSQL 16.4, run in Docker today
Search	FAISS (CPU) with BGE base English v1.5 embeddings at a pinned revision, plus BM25 implemented in the codebase
Reranking	BGE reranker base cross-encoder, pinned model revision
Document parsing	pdfplumber, with an OCR tier and a separate structured-file path
Language models	gpt-4.1-mini pinned snapshot today, with Anthropic and local model support behind one routing layer
Testing	pytest, 570 automated tests passing

11. Current status

Area	Status
Document preparation and indexing	Built and tested. Real filings indexed: five filings, three companies.
Question answering with verification	Built and tested end to end against live model calls.
Model routing layer	Built and tested.
API service and web interface	Working prototype.
Evaluation benchmark	Built. 366 hand-verified questions covering metric, temporal and entity collisions.
Structured-file path and OCR	Built and unit tested.
Tenant isolation, identity, privacy vault, escalation, audit service, connectors, research mode	Designed in detail. Not built.